HW2 - Seeded QFT / inverse-QFT Hardware Recovery

Purpose

In this assignment, you will build a personalized Quantum Fourier Transform workflow. You will prepare a seeded input state, apply QFT, apply inverse QFT, and verify that the original state is recovered. You will run the circuit on an ideal simulator and optionally run a small version on IBM Quantum hardware.

Learning goals

• Implement QFT and inverse QFT using Qiskit gates.

• Explain why QFT followed by inverse QFT should recover the original state.

• Use a personalized seed to generate a non-trivial input bitstring and measurement map.

• Correctly interpret Qiskit count keys in c[n-1]...c[0] order.

• Compare ideal simulator output with noisy hardware output.

• Export machine-readable answers.json for autograding.

Files provided

File	Purpose
student_hw2_qft_hardware_template.ipynb	Student notebook to complete and run.
schema_hw2.json	Expected structure for answers.json.
HW2_QFT_Hardware_Instructions.pdf	This instruction sheet.

Student tasks

1. Enter your student ID and generate your seeded configuration.

2. Implement QFT and inverse QFT with H, controlled phase, and SWAP gates.

3. Prepare the seeded input state in q[0]...q[n-1] order.

4. Apply QFT, then inverse QFT, then measure using the assigned measurement map.

5. Compute the expected logical qubit string q[n-1]...q[0].

6. Compute the expected displayed Qiskit count string c[n-1]...c[0].

7. Run the simulator and confirm that the dominant bitstring matches the expected displayed bitstring.

8. Record original/transpiled circuit depth and operation counts.

9. Write short reflections on QFT recovery, bit ordering, and hardware noise.

10. Export answers.json. Optionally export qft_recovery_circuit.qpy.

Important bit-ordering warning

Qiskit count keys are displayed in classical-bit order c[n-1]...c[0]. This may differ from the qubit list q[0]...q[n-1]. You must use the measurement map to determine where each qubit value is stored before forming the displayed bitstring.

```
Example:
  final qubits q[0], q[1], q[2] = 1, 0, 0
  direct measurement q[i] -> c[i]
  classical bits c[0], c[1], c[2] = 1, 0, 0
  displayed Qiskit key c[2]c[1]c[0] = 001
```

Simulator grading policy

The simulator portion is deterministic and autogradable. Because QFT followed by inverse QFT should recover the input state, the ideal simulator should place all shots on the expected displayed bitstring. The autograder can regenerate the correct answer from the same student ID and assignment ID.

Optional IBM hardware extension

If IBM Quantum access is available, run the small hardware circuit section. Real hardware results are not expected to be exact. Submit backend name, job ID, raw counts, dominant bitstring, expected bitstring percentage, and transpiler statistics.

Hardware interpretation guide

Classification	Suggested rule
Successful	Expected bitstring is clearly dominant and above 70%.
Partial	Expected bitstring is dominant but between 40% and 70%.
Inconclusive	Expected bitstring is close to other outputs or below 40%.
Failed	Expected bitstring is not dominant.

Submission checklist

- ■ answers.json is generated and downloadable.
- ■ Simulator dominant bitstring equals expected displayed bitstring.
- ■ Depth and operation counts are included.
- ■ Reflections are filled in.
- ■ Optional hardware fields are filled if hardware was attempted.
- ■ Optional QPY file is exported if requested.

Academic integrity / AI-use note

You may use AI tools to understand QFT and debug code, but the submitted numerical outputs, counts, and artifacts must come from running your own seeded notebook. Fabricated results or copied outputs are not acceptable.